

Object-Oriented Design

Lecture 03 — Advanced Programming

Ayoub FRIHAOUI

`ayoub.frihaoui@lecnam.net`

Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.
Source: `https://rezakaziz.github.io/PROG-AVANC/`

Why an object-oriented architecture?

- Organise the code so that it is **simple, clear and reusable**.
- Design classes that represent the **elements of the real world** well.
- Learn how classes **cooperate** with each other.
- Discover the **design rules** (SOLID, delegation, decoupling, etc.).

Course objectives

- Organise the code so that it is simple and reusable.
- Design classes faithfully representing the real world.
- Understand how classes cooperate with each other.
- Discover the design principles (SOLID, decoupling, delegation...).

Warning

OOP is not only a question of syntax: it is a way of thinking about the **structure of the software**, in terms of roles, responsibilities and interactions.

Useful references

- Eric Evans, *Domain Driven Design*, 2003
- Martin Fowler, *UML Distilled*, 2003;
- Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994
- Barbara Liskov, *Program Development in Java*, 2000
- Robert C. Martin, *Clean Architecture*, 2018.
- Bertrand Meyer, *Object Oriented Software Construction*, 1997
- Emmanuel Puybaret, *Les Cahiers du Programmeur Swing*, 2006
- <https://martinfowler.com/>
- <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>

The qualities of good software

Well-designed software must be:

- **Reliable** — it does what is expected of it.
- **Easy to develop** — classes are testable independently.
- **Easy to modify** — a small change does not break everything.
- **Reusable** — part of the code can be reused elsewhere.

Principles of object-oriented architecture

What is a software architecture?

Definition

The **software architecture** defines the structure of the system:

- how components are organised;
- how they interact;
- how responsibilities are distributed.

Goal

Favour clarity, **testability** and **maintainability**.

Analogy

The architecture of software is to its code what blueprints are to a building: it guides construction, extensions and repairs.

Warning

A bad architecture is expensive: fixes become risky, tests impossible, and evolutions take exponential time.

The goals of an object-oriented architecture

Main goals

- Organise the code in a **coherent and modular** way.
- Separate **roles and responsibilities**.
- Favour **reuse** and **testability**.
- Minimise dependencies between classes.

In practice

- One module = one clear business concept.
- Each class has a well-identified role.
- Interactions go through interfaces.

A well-defined class

Characteristics

A good class:

- represents a single concept;
- has a **single responsibility**;
- is **weakly coupled** to the others;
- is **highly cohesive**.

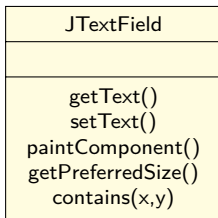
Database analogy

A well-designed class is like a well-normalised table: each entity stores coherent and independent information.

Bad sign

If a class does “too many things” (management, display, logic, storage), it is probably a God Object that must be refactored.

Example: a class that does too many things

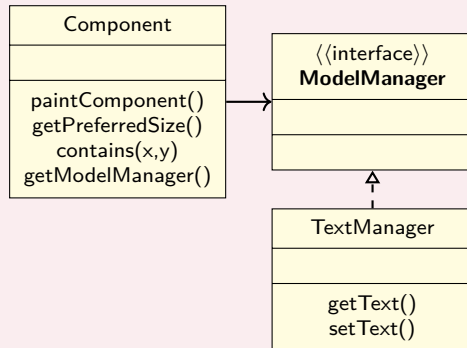


Problem: handles at the same time

- the text content;
- the graphical display;
- event management.

→ One class, three responsibilities!

Proposed refactoring



One class = one single responsibility.

Cohesion and coupling

Cohesion

Measures the **internal solidarity** of a class. The more its methods serve the same goal, the more cohesive it is.

Coupling

Measures the **external dependence** between classes. The weaker it is, the more reusable and testable the class is.

Good balance

- High cohesion: each class has a clear role.
- Weak coupling: classes communicate via interfaces.

Bad sign

A highly coupled and weakly cohesive class is hard to isolate, test or reuse. It becomes a fragility point in the architecture.

Good design practices

Essential principles

- **Simplicity**: avoid useless abstractions.
- **Clarity**: name classes after their business role.
- **Decoupling**: limit direct dependencies.
- **Testability**: design to ease unit tests.

Good reflex

Before writing a new method: ask yourself “does it really belong to this class?”

Advice

Favour **composition** over inheritance: it reduces coupling and improves code flexibility.

Abstract Data Types (ADT)

Definition of an Abstract Data Type

What is an ADT?

An **Abstract Data Type (ADT)** describes a type by **what it does**, not by **how it does it**.

- Defines the available operations.
- Specifies their behaviour (pre/postconditions).
- Hides implementation details.

Examples of ADTs

- Stack, Queue, Array, List.
- In OOP: interfaces play the same role.

Warning

The ADT does not care about the internal structure: it is a **behavioural** abstraction, not an implementation.

Goals of the ADT

Why use an ADT?

- Improve code **reusability**.
- Ease **correctness proofs**.
- Favour **client-code stability**.
- Allow several implementations for one interface.

Major advantage

The internal code can be changed without breaking the programs that use the ADT. → The contract stays unchanged.

Key principle

In OOP, objects are handled through their **interface**, not through their concrete implementation.

Formal specification of an ADT

Formal specifications

Each operation is described by:

- a **precondition** — what must be true before the call;
- a **postcondition** — what is guaranteed after the call;
- an **invariant** — what always stays true.

Description language

Mathematical logic or pseudo-code notation is often used.

Example: array of reals

Operations:

- `create(size : integer)`
- `get(i : integer)`
- `set(i : integer, v : double)`
- `size()`

Specifications:

- `create(a)`: $a \geq 0$, creates an array of size a filled with 0.
- `get(i)`: $0 \leq i < \text{size}()$.
- `set(i, v)`: $0 \leq i < \text{size}()$ and after the call, $\text{get}(i) = v$.

Abstraction vs Implementation

Separating “what” from “how”

- Abstraction describes **what** the type does.
- Implementation describes **how** it does it.
- This separation guarantees flexibility and stability.

Consequence

The internal implementation can be changed without modifying user code.

Example: Stack ADT

- Interface: `push()`, `pop()`, `isEmpty()`.
- Implementation 1: fixed array.
- Implementation 2: linked list.

The user does not need to know which one is used.

Design principle

Always **program against an abstraction**, never against a concrete implementation.

ADTs and Object-Oriented Programming

Conceptual mapping

In OOP:

- An ADT corresponds to an **abstract class** or an **interface**.
- Concrete implementations are **subclasses**.

Advantage

OOP naturally provides the encapsulation and polymorphism mechanisms needed for ADTs.

Example in Java

- Interface: `List<E>`
- Implementations: `ArrayList<E>`, `LinkedList<E>`

Methods are defined in the interface and implemented differently.

To remember

Good object design always starts with thinking about the available abstractions.

The SOLID principles

Introduction to the SOLID principles

Origin

The **SOLID** principles were formalised by **Robert C. Martin** ("Uncle Bob") to guide object design.

Goal

Create software that is:

- more **maintainable**,
- more **evolvable**,
- more **testable**.

Acronym

S — Single Responsibility Principle

O — Open/Closed Principle

L — Liskov Substitution Principle

I — Interface Segregation Principle

D — Dependency Inversion Principle

S — Single Responsibility Principle (SRP)

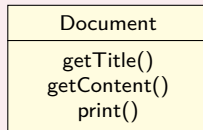
Principle

A class must have only **one reason to change**. → It must assume a single responsibility.

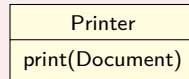
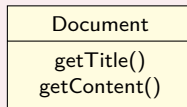
Motivation

- Limits the impact of modifications.
- Eases code understanding.
- Encourages reuse.

Example



Two responsibilities



Should rather be split into two classes.

O — Open/Closed Principle (OCP)

Principle

Software entities must be:

- **open for extension,**
- **closed for modification.**

Goal

Allow adding new features without breaking existing code.

Example

Bad: a `calculateSalary()` method with 10 ifs depending on the employee type.

Good: define an `Employee` interface with `calculateSalary()`, specialised for each employee type.

Good practices

- Clearly define the public interface of a class.
- Avoid exposing implementation details publicly.
- When relevant, provide extension mechanisms.

L — Liskov Substitution Principle (LSP)

Principle

A subclass must be able to replace its superclass without altering the expected behaviour of the program.

Statement

If S is a subtype of T, then any object of type T can be replaced by an object of type S without error.

Example

Bird class with a `fly()` method.
Ostrich inherits from Bird but does not fly.
→ **LSP violation**.
Solution: introduce a Flyable interface.

Practical rules

- A redefined method must not restrict its arguments.
- It may narrow the type of returned values (covariance).

REVIEW PREVIOUS LECTURE

I — Interface Segregation Principle (ISP)

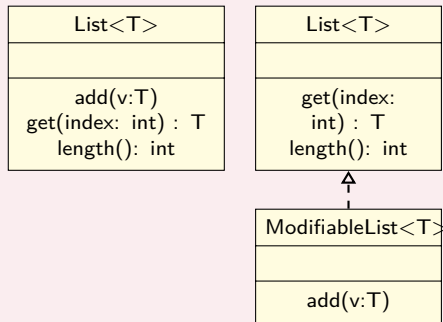
Principle

It is better to have several specific interfaces than one general, overloaded interface.

Consequence

Clients only see the methods they need. A client allowed to read a list does not need an add method.

Example



Should rather be split as shown.

D — Dependency Inversion Principle (DIP)

Principle

- High-level modules must not depend on low-level modules.
- Both must depend on abstractions.

Key idea

Code against **interfaces**, not concrete classes.

Consequence

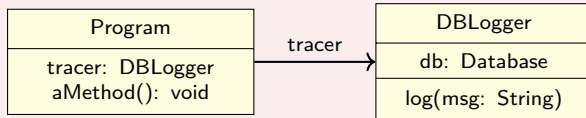
This principle favours:

- **decoupling** of components,
- **testability** (mock injection),
- **flexibility** (implementation switch).

D — Dependency Inversion Principle (DIP)

Naive design

The Program class records user actions in a database via DBLogger.



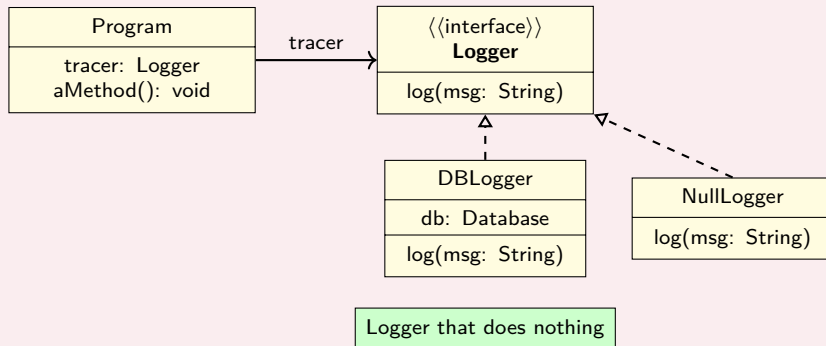
```
tracer.log("call "); ... (does something) ...
tracer.log("end call ");
```

Problems

- Impossible to test without a real Database.
- Hard to change the log destination.

D — Dependency Inversion Principle (DIP)

Solution: depend on an interface



- **Advantage:** more flexible, easily testable design.
- Program no longer depends on a concrete implementation (DBLogger).
- **Drawback:** slight additional complexity.

Summary of the SOLID principles

In summary

- **S** — One class = one responsibility.
- **O** — Extend without breaking/modifying.
- **L** — Subclasses must be substitutable.
- **I** — Small, specific interfaces.
- **D** — Depend on abstractions.

Global impact

The application becomes:

- more **robust**,
- more **evolvable**,
- more **testable**,
- and easier to **understand**.

Transition

Next, we will see how to apply these principles through **delegation** and **decoupling**.

Delegation, decoupling

Definition of delegation

Principle

Delegation consists in entrusting another object with the responsibility of performing a specific task. → An object does not do everything itself, it delegates.

Goal

- Reduce the complexity of a class.
- Favour specialisation and reuse.
- Limit coupling between components.

Simple example

A Car class does not compute speed directly: it delegates this responsibility to an Engine. `Car.getSpeed()` calls `engine.computeSpeed()`.

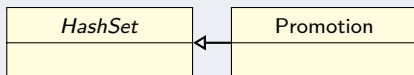
Key advantage

Delegation lets you **replace a component without impacting the others**. → Basis of substitution and decoupling.

Delegation vs Inheritance

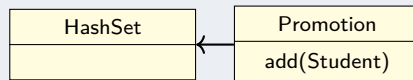
Inheritance

- “is a” relation.
- Code reuse via the class hierarchy.
- Risk of strong coupling and rigidity.



Delegation

- “has a” relation.
- An object uses another for a task.
- More flexible and modular.



To remember

Favour composition over inheritance. Inheritance expresses a strong link; delegation, a flexible one.

Reducing coupling

Definition

Coupling measures the dependence between two classes:

- Strong coupling → contagious modification.
- Weak coupling → independence and modularity.

Goal of decoupling

- Ease maintenance and tests.
- Allow evolutions without global rewriting.

UML example

If Application directly depends on Display, any GUI change affects the business logic.

Solution: introduce an IDisplay interface.

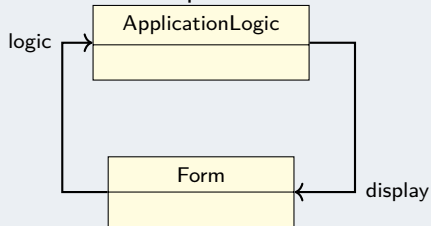
Principle

Decouple components via **interfaces** or **abstract classes** rather than concrete dependencies.

Technical removal of coupling

Typical situation

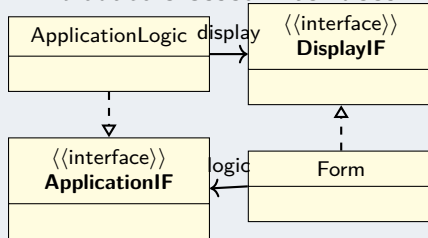
Two classes depend on each other:



→ Problem: dependency cycle.

Solution

Introduce **crossed interfaces**:



→ Each class now depends only on an abstraction.

Result

Total decoupling: each component becomes interchangeable, favouring tests and modularity.

Benefits of decoupling through interfaces

Benefits

- Less dependence between modules.
- Better testability (mock injection).
- Easier maintenance and evolution.
- Prepares multi-layer architectures (MVC, hexagonal...).

Concrete example

In a business architecture:
PatientService depends on the
PatientRepository interface.

→ We can inject:

- a PostgresPatientRepo version,
- a MockPatientRepo version for tests.

Good reflex

Always **program against an abstraction**, never against an implementation.

Advanced design principles

Introduction to advanced principles

Goal

After SOLID, other rules guide pragmatic, durable software design.

Three key principles

- **Law of Demeter** — limit indirect dependencies.
- **YAGNI** — avoid over-engineering.
- **Leaky Abstraction** — handle imperfect abstractions.

Practical stakes

These principles help to:

- Simplify architectures.
- Reduce side effects.
- Design more robust, evolvable systems.

The Law of Demeter

Principle

“Do not talk to strangers”. A method *M* of an object *O* must only interact with:

- other methods of *O*;
- methods of the parameters of *M*;
- methods of objects it creates directly.

Goal

Reduce chained dependencies and strengthen encapsulation.

Consequence

This principle improves readability and protects the code against internal structural changes.

Example

Bad: `client.getAccount().getBank().getAddress()`

Good: `client.getBankAddress()` *// the method delegates to the right object*

YAGNI — You Aren't Gonna Need It

Principle

Do not implement a feature until it is needed. (Agile methodology)

Goals

- Reduce useless complexity;
- Favour adaptability;
- Prevent over-engineering and technical debt.

Example

Bad: planning a plug-in architecture before having a single working module.

Good: implement the real need, then generalise if it becomes necessary.

To remember

Keep it simple as long as possible: useless code is a potential source of bugs.

Leaky Abstraction

Principle

A **leaky abstraction** is an abstraction whose implementation details become visible through its behaviour.

Symptoms

- Need to know the internal structure;
- Unexpected side effects;
- Client depends on technical details.

Example

Person class exposing a mutable object:

```
class Person {  
    private Date birthDate;  
    public Date getBirthDate() {  
        return birthDate; // leak!  
    }  
}
```

The caller can modify the internal date:
`p.getBirthDate().setYear(3000);`

Recommendation

Make a defensive copy:

```
return new  
Date(birthDate.getTime());
```